

triangles, as in [Figure 9-4](#). The `midpoint()` function finds the point equidistant from two points passed to the function. This will be important as the inner triangles use these equidistant points for their vertices.

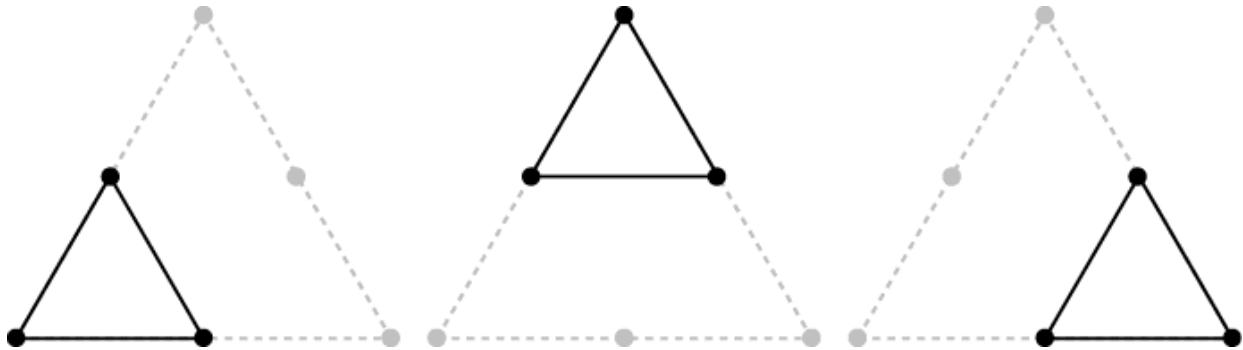


Figure 9-4: The three inner triangles, with midpoints shown with large dots

Note that this program calls `turtle.setworldcoordinates(0, 0, 700, 700)`, which makes the 0, 0 origin at the lower-left corner of the window. The upper-right corner has the x, y coordinates 700, 700. The source code for *sierpinskiTriangle.py* is as follows:

```
import turtle
turtle.tracer(100, 0) # Increase the first argument to speed up the drawing.
turtle.setworldcoordinates(0, 0, 700, 700)
turtle.hideturtle()

MIN_SIZE = 4 # Try changing this to decrease/increase the amount of recursion.

def midpoint(startx, starty, endx, endy):
    # Return the x, y coordinate in the middle of the four given parameters.
    xDiff = abs(startx - endx)
    yDiff = abs(starty - endy)
    return (min(startx, endx) + (xDiff / 2.0), min(starty, endy) + (yDiff / 2.0))

def isTooSmall(ax, ay, bx, by, cx, cy):
    # Determine if the triangle is too small to draw.
    width = max(ax, bx, cx) - min(ax, bx, cx)
    height = max(ay, by, cy) - min(ay, by, cy)
    return width < MIN_SIZE or height < MIN_SIZE

def drawTriangle(ax, ay, bx, by, cx, cy):
    if isTooSmall(ax, ay, bx, by, cx, cy):
        # BASE CASE
        return
```